The interactive speech applications platform

P M Hughes, P B Quilliam and K R Rose

The interactive speech applications platform (ISAP) is currently used by BT to host the CallMinder, Automatic Customer Interface and Automatic Meter Reading (MeterLink) services along with further services to be launched shortly. It has provided the vehicle for a number of advanced application trials both within the UK and in the United States. The integration of these new services within BT's network, and the underlying design philosophy for the ISAP have been described previously [1]. This paper explains in more detail the architecture of the ISAP and the reasoning behind some of the key design decisions. The way in which the platform is evolving to support increasingly advanced applications is reviewed.

1. Introduction

Following an evaluation of internationally acclaimed speech platforms the interactive speech applications platform (ISAP) was chosen as the preferred platform for the roll-out of CallMinder and for future BT voice services. Originally designed and developed by BT, the ISAP is licensed to Ericsson Ltd who are now design authority for the platform. Indeed, a number of the applications hosted by the platform are now being produced and supported by Ericsson. Although the ISAP was designed primarily for voice applications — hence the reference to 'speech' in ISAP — it can also support non-voice applications, e.g. MeterLink.

While some of the platform software development and design support is still undertaken by BT, under subcontract to Ericsson, a programme of technology transfer between the two companies is actively extending Ericsson's capability [2]. The ultimate aim is to equip Ericsson with the skills and knowledge to onward develop the ISAP platform itself and to enable them to produce new interactive speech applications for BT and the world market.

A high-level review of voice services and the way in which the ISAP has been developed to meet the needs of such services, together with a review of the functionality offered by the platform, has been covered recently [1]. In this paper, some of the underlying design philosophies associated with the ISAP are examined further, along with the relationship between the hardware and software elements of the platform. Other related aspects, such as service creation tools, are covered elsewhere [3].

2. Architectural overview

2.1 Design philosophy

The top-level design objectives for the ISAP may be regarded as providing the means for ensuring the cost effectiveness and fitness for purpose of the platform:

- cost optimisation — to ensure that the platform was cost effective over a range of different service types and complexities,
- network worthiness — addressing security, resilience, management and environmental factors to meet the operational requirements of BT's network,
- future enhancements — ensuring that the platform architecture and design was sufficiently flexible to meet future advanced service requirements and could benefit from the continuing improvements in price/performance of advances in technology.

These points were addressed by three key design decisions:

- use of industry and open standards wherever possible,
- modular design within both hardware and software,

- a common hardware base allowing a highly efficient dynamic allocation strategy to be used for speech/ signal processing resource management.

The wider implications of these design decisions are reviewed in the following sections.

Industry and open standards

The ISAP makes extensive use of industry and open standards, both hardware and software. This approach was adopted in order to reduce the risk and the time taken to develop the platform. In addition, it allows the platform to be upgraded easily as enhanced hardware and software become available. The quick and successful transfer of platform manufacture and support to Ericsson Ltd was a consequence of this key design decision.

The key standards which were adopted for the ISAP are VME bus for hardware systems and UNIX, or UNIX-like equivalents, for the software systems. The rationale behind these choices is covered in more detail later in this paper.

Of the main hardware card types used in the ISAP, only two are unique to the platform — the speech processing card and the digital line interface card. At the time the ISAP was developed, no proprietary equivalents which could provide these functions were available. However, recent information from various suppliers suggests that even these cards might soon be superseded by industry offerings.

Modular design

The ISAP is an integrated system comprising five basic component types connected together over a local area network (LAN) (see Fig 1). The combination of the components required for a given application system will depend on the specific nature of the service and the anticipated traffic.

Communication between components or sub-systems across the LAN is conducted according to a common (proprietary) message protocol. This allows the individual functional modules to be developed or enhanced/modified in isolation, simplifying and speeding up the development process.

Further modularity is obtained by using common hardware building blocks within components. This allows the spares holding required to be reduced and provides for greater economies of scale during manufacture and procurement.

The modular design approach coupled with the use of industry standard hardware and operating systems has also provided a direct route to upgrading the performance of the

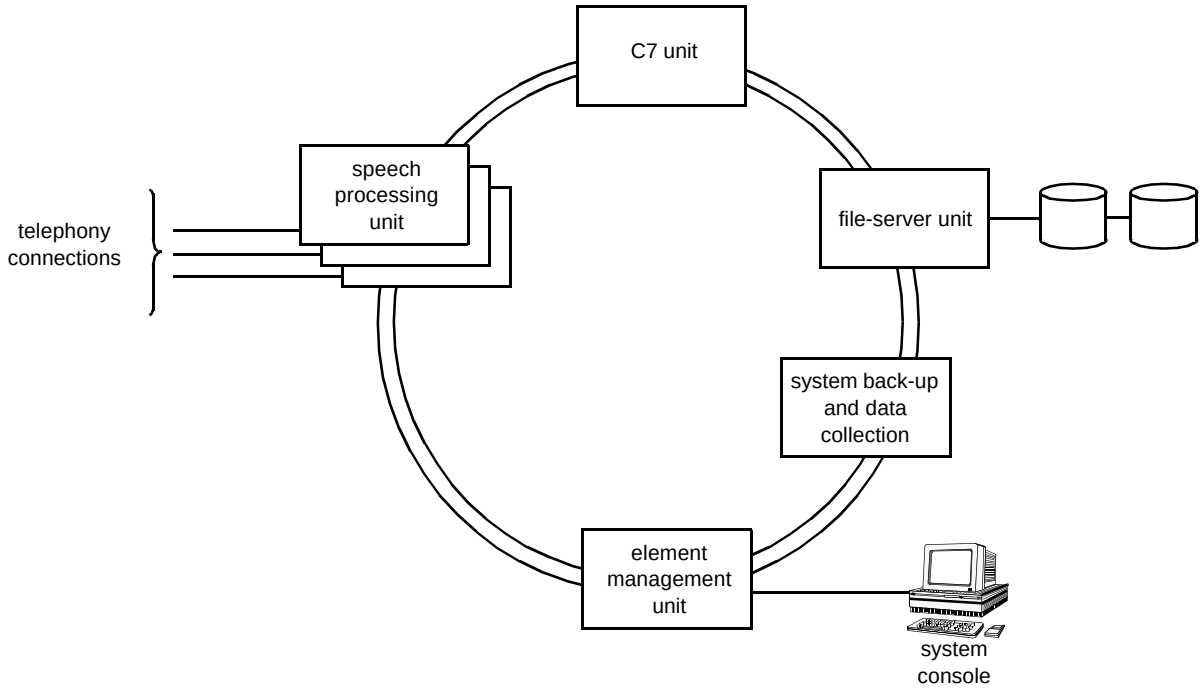


Fig 1 The ISAP integrated system showing the five basic component types.

ISAP without having to undertake extensive system or sub-system re-design.

Dynamic speech resource allocation

At a basic level, a voice application consists of a sequence of speech processing functions or algorithms. These different functions require differing levels of processing capability, e.g. a large-vocabulary speech recogniser is much more computationally demanding than a TouchTone[1] detector. In addition, different services or applications use the various functions in differing proportions. For example, leaving a message on the CallMinder call completion service does not require the use of speech recognition, while retrieval of messages by the mailbox owner does. Hence, the overall level of processing power required will vary from application to application and will also vary dynamically within an application, depending upon the particular part of the application being exercised.

Until recently, the majority of speech platforms were designed such that the processing resource was fixed and divided equally across all the input channels. In order to account for 'worst case operation' this resource level has to accommodate the most complex algorithms in the most resource hungry combination that might be encountered (e.g. simultaneously perform a speech recognition operation, record and store the user response and in parallel 'listen' for a TouchTone interrupt). However, for the majority of time during the execution of an application, the most computing intensive algorithms or combination of algorithms will probably not be accessed, and hence this expensive processing power lies unused for much of the time. Similarly, the fixed resource allocation sets a limit on the maximum complexity of the speech processing resource any particular channel on a platform can support, without reconfiguring (or potentially redesigning) the system.

The ISAP is designed to overcome this drawback by dynamically allocating speech/signal processing resources to the input channels from a common resource pool. The foundation for this technique is that all algorithms/functions are software based and can run on a common hardware base. Resource control is achieved by a combination of dynamically switching incoming ports to processors where a required process is already loaded but is not in use, and by dynamically managing the loading of the software-based processes into the processor pool to match the total requirements on the platform at any particular time. With this technique there is potentially a risk that a particular resource will not be available when required, leading to the failure of the application. This risk can, however, be minimised by using pre-emptive scheduling (for example, a speech recognition operation is always preceded by playing out a prompt, so giving time to find and allocate the recogniser), a retry strategy and software-controlled locking of critical resources.

The processors involved are themselves programmed to provide single-function, multi-channel operation (i.e. running multiple instances of the same signal processing function), which provides for highly efficient usage and reduces software integration problems as new features are added to the platform. The end result is that the platform's processing resources are managed to function at maximum efficiency. Clearly there is a consequent cost benefit which arises from not having under-utilised computing capacity lying idle for much of the time.

Increasingly sophisticated signal processing and manipulation processes demand ever-increasing levels of processing capability. The use of a common pool of processing resource allows these increasingly complex algorithms to be accommodated on the ISAP without having to undertake a major re-work of the existing system software. As the algorithms are dynamically loaded from external memory/store they can be installed or upgraded in a relatively straightforward manner, minimising platform down time — an essential feature for operational systems.

2.2 Deriving the ISAP system architecture

In any system of this complexity, it is inevitable that a number of design issues arise. For example, the way in which different tasks or computing processes are distributed over the different processors in the system is a key issue. In practice, with ISAP there were two levels of design decision to be made, the separation of the overall system into modules or sub-systems and then the design of the individual sub-systems themselves.

The arrangement of sub-systems that was finally selected for the ISAP hardware architecture centres around five main sub-systems and an interconnecting local area network as shown in Fig 1. The high-level functions provided by each of the modules are:

- the signal processing unit — performs all the signal processing functions and is responsible for running the software that controls the application,
- the signalling unit — provides the platform data control interface to external systems, e.g. in the case of the BT telephone network this unit would use the ITU No. 7 signalling protocol,
- the fileserver unit — a general filestore function also used for storing/retrieving stored speech messages, etc,

[1]TouchTone™ is transmitted by pressing the keypad on modern 'tone' sending telephones.

- the element management unit — provides system and service management functionality for the platform and is the gateway to external systems,
- the system back-up and data collection unit — performs all back-up and event log archival,
- the local area network — interconnects all the above units to form the overall system.

This particular arrangement is a fairly natural task-based breakdown of the system components which allows the individual specialist modules to be developed without having to make unnecessary compromises.

2.3 Sub-system architectures

A high-level description of the functions of the various sub-systems shown in Fig 1 can be found in Rose and Hughes [1]; however, the signal processing unit is worthy of further, more detailed consideration since it is the heart of the platform. The ISAP local area network is also described as it the method by which all the ISAP units are connected together.

2.3.1 Signal processing unit

The signal processing unit is the core application or dialogue engine of the platform. It has to perform all signal processing functions along with the high-level signalling functions required to allow the caller and platform to interact. All of these operations have to be performed in real time, yet the platform has to provide the developers of new applications and services with straightforward, simple-to-use commands. The application developer, for example, simply wants to be able to request such things as the replay of a speech prompt or the execution of a speech recognition, or maybe direct the platform to answer an incoming call, all without having to be concerned with all the internal data transfers, system control, signalling and resource management issues which these commands entail. The design must therefore take into account these system ‘usability’ issues.

Within the signal processing unit, a number of the design decisions were promoted by the original philosophy of using industry or open standards.

In terms of the hardware, it had been decided that the VME-bus was the most appropriate industry standard to use for the platform. The decision to use VME-bus provided access to a wide range of different single board computers which could be obtained readily on the open market. In addition, the VME market is well populated with suppliers and is highly competitive in terms of price/performance for such modules. Such a competitive market also ensures that as technology develops or new microprocessors become available, then the new VME modules which make use of these new devices appear shortly after. This provides an efficient and highly cost-effective route to system and performance upgrades.

Although the VME-bus has a relatively high capacity and can support very high-speed data transfer in burst mode, handling multiple real-time interactions is very demanding in terms of data transfer, particularly when multiple streams of speech data (e.g. messages stored on CallMinder) might also have to be transferred to and from external storage systems. As such, and in order to minimise the possibility of the VME-bus becoming a system bottleneck, it was decided that a second bus should be used to transport the signal data, e.g. 64 kbit/s digital speech, from the network interface to the processors which perform the signal processing operations, etc, and vice versa. This bus, known as the speech bus (Fig 2), is capable of handling 120 × 64 kbit/s pcm time slots.

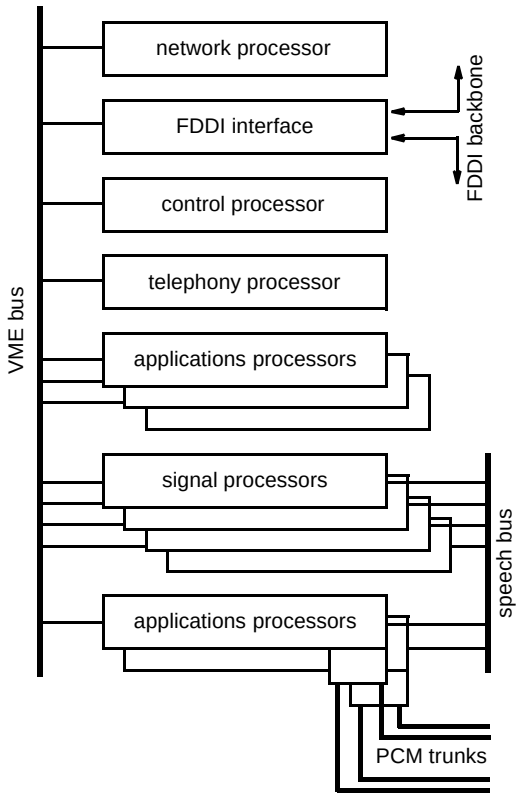


Fig 2 Signal processing unit architecture.

The rationale behind the general architecture of the signal processing unit (Fig 2) was to separate the execution of the application control software from both the internal control processes and the execution of the signal processing algorithms. This is achieved in hardware terms by using different physical processors, each processor type selected by matching the hardware to the software task to be

undertaken, but bearing in mind that proliferation of different card types impacts directly on support and maintenance costs because of the spares holding required.

In software terms the separation is achieved by running the application control software under Unix, while the real time operations on the control and speech/signal processing cards utilise VxWorks, a Unix-like operating system designed specifically for real-time operation. This separation of application and real-time signal processing functions also helps to provide a 'fire wall' between the application and the internal operation of the platform, so facilitating a more straightforward approach to the development of the applications software. In addition, it simplifies maintenance, support and testing activities, by helping to ensure that the core system cannot be violated by the application software.

The various processors which make up the signal processing unit are as follows.

- The digital line interface card (DLIC) — this card was one of the two major cards specially designed for the ISAP. It provides the basic connection between the platform and an external network, for example, the telephone network. For telephony-based applications each DLIC supports two CEPT-30-type 30-channel pcm digital telephony connections and each signal processing unit can support up to two DLIC modules (i.e. a total of 120 telephony channels over four CEPT-30 spans). As the VME-bus does not physically meet the regulations for equipment to be connected directly to a telephone network, a separate isolating barrier board, mounted on the back of the VME-bus, is used. This allows for the network regulations which apply in different countries to be accommodated by simply changing the low-cost, passive-barrier board, rather than requiring country-specific DLIC cards. The line card indicates loss of clock on incoming trunks with a visual alarm, and will automatically attempt to synchronise to a working network connection.

  The incoming 64 kbit/s pcm speech data is switched on to the speech bus by the DLIC by means of a time/space switch. This allows incoming speech data to be connected to any available time slot on the internal speech bus. Similarly, outgoing pcm speech data can be received from any designated time slot on the speech bus and routed through to any specific telephony channel on one of the CEPT-30 channels.

  With regard to signalling, in the default state, the card will support channel associated signalling (CAS). Common channel signalling (CCS), however, may be supported by routeing signalling data on nominated line interface cards within a system, over a V.11 data link, to the signalling unit (Fig 1) where further processing is carried out.

  For connection to non-telephony networks, the DLIC would be replaced with an appropriate interface card, e.g. an ATM interface card.

- The signal processor — the ISAP signal processor card is the second of the major cards designed specifically for ISAP. This card hosts all of the signal processing algorithms. These run mainly on Motorola DSP56001 digital signal processors (four per card), each with 128 Kbyte of fast (zero wait state) memory. A 68EC030 microprocessor, floating-point co-processor and 4 Mbyte of memory are also provided. As the signal processor card is used in relatively high volumes within an overall system, this particular combination of processors was chosen very much with cost in mind. The DSP itself is a fixed-point device. This creates additional complications for the algorithm developers who have to take into account not only the calculations which comprise the algorithm, but also the scaling of the digital signal to avoid problems with signal-to-noise ratio, overflow and underflow, etc. Some of the algorithm calculations simply have to be performed as floating-point operations and hence, for computational efficiency, a floating-point co-processor was included. Overall control is provided by the 68EC030. This particular device was selected to some extent because of the ease with which it can be integrated with the co-processor; however, the primary deciding factor was the need to achieve a relatively high packing density on the card — this particular processor was available in a small outline package at reasonable cost.

  Each of the DSPs on the card has a 30-channel interface to the speech bus through a time/space switch, allowing any time slot on the speech bus to be switched to any channel on any DSP. This mirrors the function of the time/space switches on the digital line interface card(s) and provides a high degree of flexibility.

  The capability and number of signal processors required in an installation depends on the statistical application dialogue coverage.

- The network processor — the network processor (NP) is a standard VME single board computer, currently a card based on the Motorola 68040 microprocessor, although competitively priced 68060-based systems are now emerging. The processor performs the system controller function as defined in the VME bus specification.

  The ISAP system software makes the VME-bus back-plane appear as a distinct local area network (isolated from the LAN which interconnects the various sub-systems). Hence, a key role of the NP is to control the back-plane network and provide a gateway between the ISAP signal processing unit and the rest of the system.

- The control processor — the control processor, again a standard single board computer, runs the main ISAP signal processing unit control software and is responsible for download and control of the algorithm software running on the signal processing cards. It is this processor which controls the dynamic allocation of signal processing resources and it is this card, therefore, which is responsible for monitoring and maintaining a table of how all the digital signal processing devices on the signal processing cards are configured and which DSPs are currently in use.

  The control processor also hosts the shelf alarm handler and is configured to be able to reset the shelf when instructed to do so.

- Telephony processor — the telephony processor, another standard VME computer card, converts the signalling information received from the signalling unit into messages that can be easily controlled via the application program and vice versa.

- The applications processors — the applications processors are responsible for running the application; it is the application software which determines the processing functions which are to be executed and the order in which they occur. As noted, the applications software runs under the Unix operating system (currently SUNOS4.1.3/Solaris 1.1). At present, the applications processors themselves may be any one of the SPARC-based processor family which runs this operating system. The decision to use SPARC-based cards was made on price/performance and a visible upgrade path, as the capability and capacity of the processors improved over time. In addition, the SPARC family provides a high degree of backward compatibility across processors. The processing requirements for the card in any given application depend heavily on the number and type of applications to be run on the applications processor card.

  The applications processor does not use the VME bus back-plane for communication. All communication is via ethernet to the network processor. In fact, this processor does not have to be physically installed within the signal processing unit, it needs only to be able to communicate with the network processor. This flexibility has been used to good effect within the service node architecture [4].

- FDDI interface — the FDDI interface card connects the shelf to the system local area network. A network driver for the card is hosted by the network processor.

2.3.2 ISAP local area network

The local area network is the means of interconnecting all the ISAP units or sub-systems as shown in Fig 1. The network is based around the Internet protocols (IP) running over an FDDI optical fibre network. All nodes in the LAN have dedicated network numbers, and all nodes are configured as routeing gateways. Each node on the LAN has a gateway data transfer bandwidth of approximately 300—400 kbyte/s.

FDDI-based networks are generally used for high capacity systems, or for systems where resilience is of paramount importance. Both of these requirements apply to the ISAP. A dual-attached FDDI ring, as used on the ISAP, can suffer a complete failure in one of the rings without loss of data. Should this occur, alarms are provided to allow for prompt repair. The FDDI ring is token based, so the full network bandwidth is available. This order of bandwidth is desirable when speech messages are being deposited and retrieved, as they are on CallMinder and equivalent systems. The original choice of an FDDI LAN, made at the outset of the project, was specifically driven by the desire to avoid the system ‘backbone’ becoming a bottle-neck or a barrier to system growth.

2.4 Specifying a signal processing unit

A direct result of the versatility and highly configurable nature of the signal processing unit is that the process for specifying the type and number of cards within a unit becomes an interesting and application-dependent problem. The hard limits are the speech bus, which can carry up to 120 pcm time slots and the VME shelf which can house up to 21 cards. As a minimum, a signal processing unit will generally require one digital line interface, one network processor, one control processor, one telephony processor and one FDDI card (although there are exceptions to this when fewer cards can be used). These cards take up five of the 21 slots leaving a further 16 slots which can be populated with applications processors, signal processors and possibly an additional line interface. The relationship between the number of application processors and the number of signal processors is essentially application dependent. For example, running multiple instances of an application which makes heavy use of speech recognition will require far more speech processor cards than an application which uses only TouchTone signals to collect input from the caller. Similarly, it is possible to run more instances of a simple application on an application processor than applications which place significant demands on the application processors (e.g. applications involving local database searches, etc). As such, it is necessary to identify the optimum balance between application processors and signal processors which ensures that the computing and signal processing resource is used efficiently but without reaching the point where either the application processors start to give performance problems when under heavy load, or where the speech processing resources are not available when required by an application.

3. ISAP Software

Although the preceding sections relate mainly to the ISAP hardware, it can be seen that it is difficult to separate the discussion of hardware and software architectures. However, as with the ISAP hardware, there were certain key drivers which influenced the general approach to the software architecture which evolved. One such was the desire to simplify the applications development process as much as possible, so reducing the costs associated with new application developments, but, perhaps more importantly, reducing the time to market. As such, the applications developer's view of the ISAP is a set of high-level object oriented applications programmers interfaces (APIs). As a result, the application developer is intentionally divorced from the inner workings of the platform. The nature of the ISAP APIs, along with the general nature of the other platform software is examined further in the following sections.

3.1 The ISAP application programmers interface (ISAP-API)

An object oriented (OO) approach was selected as the basis for the ISAP API as this model generally lends itself better to the complex constructs which are used in applications. In addition, the OO approach promotes reuse of modules which may already have been developed and provides a level of abstraction from the platform. The latter is an important feature as it allows the platform software to be upgraded without affecting the application software. This facility also allows changes to be made to the underlying signalling system (e.g. changing from ITU-7 signalling to DPNSS, a private network signalling protocol) without having to make extensive modifications to the application software. Finally, the C++ language used by the ISAP is commonly used internationally and hence there exists a human resource pool which can be drawn upon without having to provide extensive special training.

The ISAP API itself is an object oriented C++ class library which provides the necessary primitives that automatically control the ISAP. In addition, constructs to allow complex combinations of operations can be supported.

For example, in a speech application running in a telephone network, the basic classes provided, as shown in Fig 3, are:

- telephone class — allows telephony control of the associated ISAP port — basic functions are AnswerCall(), MakeCall(), and ClearCall(),
- prompt class — allows encoded speech files to be opened and concatenated using Append(), and played out on an active telephone object using Play(),
- recording class — transfer a specified length of speech to a disk file using Record() — set silence detection using SetPreTimeout(), SetPostTimeout(),
- vocabulary class — Open() a specified speech recognition vocabulary, and Recognise() on an active telephone object,
- TouchTone receiver — set NumberOfDigits() and silence time-outs, and Receive() on an active telephone object.

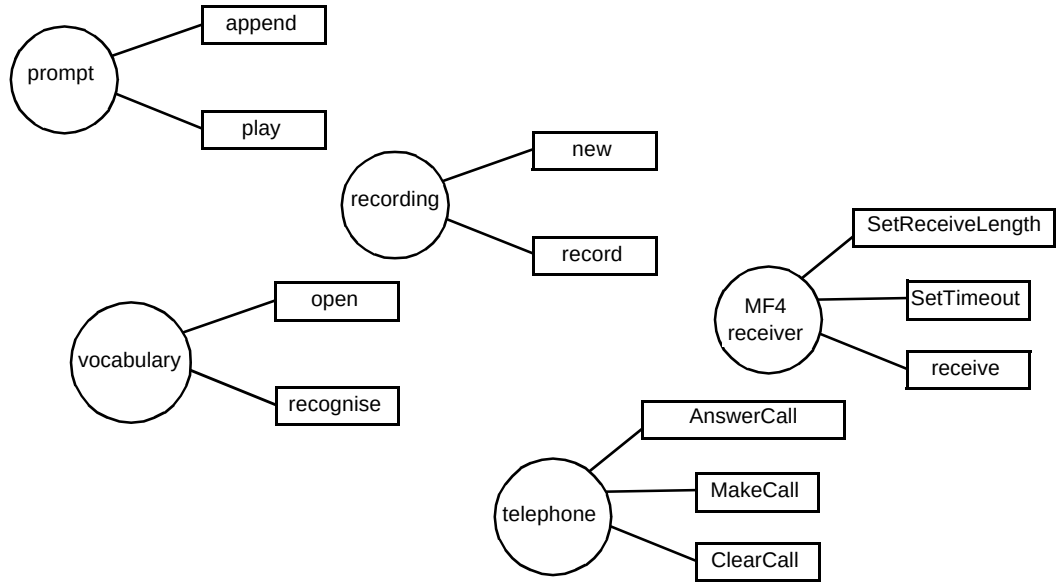


Fig 3 The basic classes provided in a speech application.

In addition, derived class hierarchies are defined which provide for such things as specific telephony-signalling functions and speech-recognition algorithms.

The basic classes allow the construction of simple dialogues, while the action functions cause request messages to be sent to the real-time part of the ISAP to request the appropriate resources. When a request has been serviced, the status of the requested operation at completion is returned. In many applications, however, more complex operations are required such as simultaneous speech recognition and TouchTone digit entry (e.g. as used in Call Minder message retrieval where TouchTone '1' and '2' may be used in place of saying 'yes' or 'no') or the recording of the spoken utterance to a recogniser for later analysis or audit. The SAP-API provides the 'node element' for this purpose.

The node element provides a template to combine the operations of a number of basic API objects into complex operations to be used at a particular 'node' or junction within an application. A typical template is shown in Fig 4. In this example, individual speech operations are attached to the node element using Set() functions, causing slots within the template to be filled. When required, the node element Do() function is called on as an active telephone object, which results in the associated request messages being sent to the ISAP real-time part. Here the ISAP application control layer manages the necessary resource requests and the real-time interactions between node operations, for example, executing the speech recogniser as soon as the prompt play has completed.

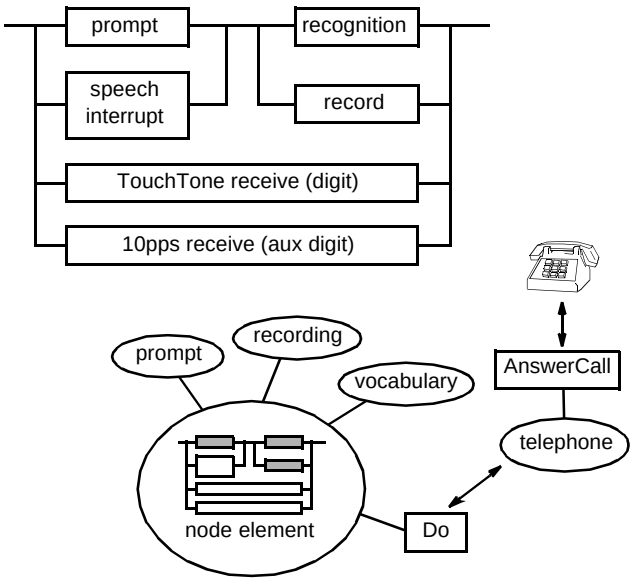


Fig 4 A typical node element template.

Using object oriented principles, it is then possible to build classes from a number of node elements which implement a multi-layered dialogue node whereby increasingly verbose prompts are played if the individual node elements fail to elicit the desired information. For example, an initial prompt to request the replaying of a recorded message may be: 'Play message?' If no response is detected, a more verbose prompt such as: 'Answering after the tone with yes or no, would you like to play that message?' These macro-nodes are referred to as dialogue building blocks, and form the basis of the implementation of the BT Voice Applications Style Guide for interactive speech dialogues on the ISAP [3].

It should also be noted that these building blocks are themselves templates consisting of:

- the names of the prompts used at each level,
- the name of the speech recognition vocabulary used,
- the number of TouchTone digits expected.

The provision of the above parameters to these templates can be automated by use of a graphical user interface (GUI) as screen-based forms. When these input forms are combined with a flowcharting package which describes the control links between these dialogue blocks, the end result is a dialogue creation tool. This is the essence of the Visage service creation tool [3], which allows a dialogue structure to be entered via an easy-to-use GUI, and when completed, the corresponding API C++ code to be generated. This code manipulates the dialogue blocks, and hence at run time causes the ISAP to be controlled via the API.

3.2 The ISAP management programmers interface

In addition to controlling the application, the application developer needs to manage and control the ISAP. This capability is provided by means of the ISAP management programmers interface (MPI).

The ISAP-MPI is designed using similar principles to the ISAP-API, but here the classes are used to represent the physical hardware and system control functions of the ISAP. Example classes are:

- signal processor class — functions are provided to SelfTest(), Reboot(), GetStatistics(),
- telephony processor class — GetChannelStatus(), GetCallStatistics(),
- ISAP shelf class — allows control of the appropriate number of processor objects in a ISAP signal processing unit,

- ISAP alarm handler — allows collection of alarms from a single ISAP unit for routeing to network management centres.

By using the above, it is possible to provide an application which both executes the chosen dialogue efficiently and provides the necessary system and service management facilities for control and monitoring of the service.

3.3 Looking behind the APIs

While the ISAP APIs form the interface to the application developer, a great deal of software underpins the various dialogue and management functions which can be invoked. In general software terms, the ISAP can be considered as a hierarchy of components. These are:

- signal processing functions — this component covers all the signal processing functions of the ISAP, the external interface to this component being the ISAP system application programmers interface (part of the ISAP-API),
- signalling interface functions — this component covers all signalling systems supported by the ISAP, the external interface to this component being the ISAP telephony application programmers interface (part of the ISAP-API),
- management functions — this component covers the functions required to manage the operation of the ISAP (note this does not include any service specific functionality), the external interface to this component being the ISAP system management programmers interface (ISAP-MPI) which incorporates telephony commands,
- generic software — this is software that can be reused between applications, for instance the generic platform manager, application management and archiving,
- operating systems — the basic operating systems used on all the ISAP processors.

Each main component is further subdivided into smaller components which perform specific tasks, e.g. the speech resource allocator. All ISAP components communicate with one another using a message-based protocol known as the ISAP inter-process communications mechanism.

As noted, the applications software and the core platform software are separated by a firewall through which the only communication is by means of the ISAP APIs. There is a further separation of application and core platform software brought about by the operating systems used. The applications software (running on the applications processor) runs under the Unix operating system, while the majority of the other software (running on the network processor, control processor, telephony processor and the speech/signal processors) runs under a Unix-like real-time operating system (VxWorks). The reason for using two different operating systems relates to the nature of the tasks performed by the different elements of the systems. To aid ease and speed of development, application software needs to be very high level while the core platform software needs to be highly efficient and capable of managing the complex real-time interactions

In order to execute a simple ‘speak prompt then perform recognition’ command, say, a great deal has to happen in platform control and signal processing terms. Nearly all of the associated actions are time critical as delays in execution can lead either to a delay to the start of a message being played out, interrupting the flow of the dialogue, ‘gapping’ within the message itself, or delays in a speech recogniser being available to collect the user response. For this reason the real-time operating system, VxWorks, was chosen as the environment for the core platform software.

VxWorks offers a number of valuable features which facilitate real-time operation of the platform software. It combines a proven real-time kernel with standard Unix networking features such as TCP/IP and NFS. It also provides a POSIX-compliant programming interface, which promotes easy migration of speech algorithms and control software from a Unix development environment to the final run-time processor.

4. Testing the ISAP

The complexity of the ISAP is broadly equivalent to that of a network switch. The design for testability and actual testing processes for such a complex real-time system is a subject which merits a far more complete treatment than is appropriate to this paper. However, a short overview is presented here to provide some of the background to this subject.

Testing and testability are factors which arise not just at the completion of development but are an inherent theme throughout the development life cycle. It is vital that components, modules, etc, are designed with testability in mind and that testing takes place throughout the development process.

At component level, testing is generally carried out using test harnesses which run on local workstations or small ‘stand-alone’ VME-based development systems; a further advantage of the VME approach is that small development systems can be produced at relatively low cost. Integration and system level testing is performed on ISAP models. The models are configured to be representative of target ISAP installations, complete with all communications and management interfaces.

The majority of system tests are run automatically through a combination of shell scripts running on Unix workstations and PC-based test automation tools. Outputs from configuration management and code analysis tools provide information to the automated test environment which identify the minimum test set which needs to be run to test out the most recent set of software changes.

Variable load and prolonged running tests are executed by a system known as the CallProccessor (CAPROC). This system is capable of simulating the user aspect of interactive speech and/or tone dialogues. It is essentially an ISAP system which is connected 'back to back' with the model under test.

While extensive testing has been an inherent part of the ISAP development, it has to be acknowledged that ultimately in systems of this complexity, whilst testing provides a valuable simulation of possible system usage, it is not a substitute for active field testing with real users. As such, piloting new applications prior to full launch must also be a key part of the overall verification, validation and testing strategy.

5. Futures

At present, the main drivers for further ISAP developments are the need to provide greater functionality to support new and more advanced services, while undertaking cost reduction through substitution of more recently available processing cards which provide a better price/performance. In particular, interest is centred on equipping ISAP with large vocabulary speech recognisers (2000 words or greater) in a cost-effective manner. This requires a slight change in the design philosophy, since the computational demands of these recognition algorithms exceed the computer power available on the existing ISAP speech/signal processing cards.

However, in practice, the versatility of the system architecture is such that this has not proven to be a major problem. Indeed, systems, where the 'front-end' signal, processing aspects of the new recognition algorithms are executed on the signal/speech processor cards, and the computationally demanding back-end operations are executed on high-performance general-purpose processors, have already been trialled successfully in the United States. The next step in this process is to find ways of improving the computational efficiency of this arrangement in order to reduce the cost of providing this large-vocabulary recognition. Optimising the management and dynamic resource allocation of the extended resource pool created by this separation of tasks is also an interesting issue.

A further area of ongoing development relates to the running and management of multiple (different) applications on the same shared speech processor units. The ability to host multiple applications can potentially reduce equipment costs when the traffic profiles for the different services are complementary, However, in practice, the usage patterns for voice services often tend to be quite similar. As such, the real advantage to be gained from running multiple applications is that the infrastructure cost can be amortised over a bundle of services providing a cost effective route for the delivery of low-volume or more marginal services. In order to facilitate this, a software framework is being developed to allow applications to be dynamically loaded at run time, depending upon information received over the signalling links.

While the main application demands on the ISAP for the foreseeable future are in the voice domain, the platform architecture is equally applicable to multimedia services. It is hoped to shortly investigate the implications of running multimedia applications on a common ISAP to voice services.

6. Conclusions

The ISAP is a significant development for BT and for Ericsson. It has broken new ground both in the technical approach which has been adopted and in the commerical and technology transfer relationships which have been established. That the same basic system can be used to host the variety of different applications which are covered elsewhere in this issue is an active demonstration that the design approach has delivered a versatile yet usable system. With interest in the ISAP now extending world-wide through Ericsson Ltd, the ISAP is set fair to continue its evolution and so serve BT's needs for some time to come.

References

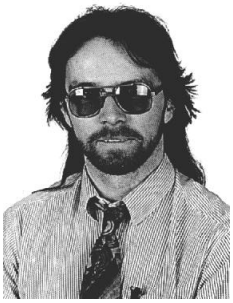
1 Rose K R and Hughes P M: 'Speech systems', BT Technol J, 13, No 2, pp 51—63 (April 1995).

2 Twist P, Briggs T, Morgan K and Hughes P M: 'ISAP — delivering the goods', BT Technol J, 14, No 2, pp 35—42 (April 1996).

3 Chester T, Hanes R and Cheshire J: 'Service creation tools for speech interactive services', BT Technol J, 14, No 2, pp 43—51(April 1996).

4 Kabay S and Sage C J: 'The service node — an advanced IN services element', BT Technol J, 13, No 2, pp 64—72 (April 1995).,

Pat Hughes graduated from Liverpool University with a BEng in electronic engineering in 1978 and went on to study for a PhD. Some six years later, having completed his doctorate in digital signal processing and spending three years as a lecturer, he joined BTL, working on speech analysis and synthesis. He became involved in the development of speech platforms working initially with network-based message announcement systems. Later, he worked on the initial design of the SAP and then co-ordinated its onward development. He is now in the Corporate Programme Office managing BT's Strategic University Research Programme.



Phil Quilliam graduated from Leeds University in 1986 with a BSc Hons in Computational Science, and joined BT Laboratories software engineering division working on Unix kernel programming for the Mezza voice system, and real time software using Martello controllers.
He joined the speech application division in 1990 and was responsible for the software architecture design of the ISAP. He currently leads a team producing advanced ISAP applications, which has worked closely with MCI on a number of service trials.

Kevin Rose joined BT Laboratories (BTL) as an apprentice in 1973. After graduating from Essex University with a BSc in computer systems and an MSc in telecommunications systems in 1983, he joined the newly established speech division in BTL. He has remained in the field of speech technology except for a short period in BT's switching systems business unit. In 1990 he became responsible for the development of the BT SAP and has managed its development from feasibility through to implementation into BT's telephone network. This has included managing the launch of the CallMinder pilot service. He recently became manager of BT's range of video-telephony products, as well as the application of the SAP in advanced voice trials.